

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT**

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	C Shahanaj	Reg. No.:	192372292
Section / Batch:	CSE-AI /2023	Date:	24/07/2026

Instructions to Students

- Answer BOTH scenario-based questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – SCENARIO BASED ASSESSMENT

Q1. A code editor searches for variables and keywords in large programs using brute-force string matching.

- Explain how the algorithm works.
- Analyze its time complexity.
- Justify the need for optimized string searching methods.

Code of Conduct

I ___Shahanaj (192372292)___(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___Shahanaj_____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25	Thoroughly understands the scenario	Clearly understands the scenario	Basic understanding; some aspects	Poor understanding or
& Context Analysis		with accurate interpretation of all requirements	with minor gaps	of the scenario unclear	misinterpretation of the scenario

Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Question No.	Problem Understanding & Context Analysis (25%)	Application of Concepts (30%)	Analytical & Decision-Making Skills (20%)	Solution Design & Approach (15%)	Clarity, Justification & Presentation (10%)	Total Marks (Each – 50)
Q1	4	3	3	2	4	40
Q2	4	4	4	3	2	46

Overall Total (100)	86
----------------------------	-----------

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. A code editor searches for variables and keywords in large programs using Brute-Force String Matching.

a) Explain how the algorithm works.

Introduction

String matching is the process of finding whether a pattern (P) exists inside a text (T). In a code editor, this technique is used to search for:

- Variable names
- Function names
- Keywords (if, while, return, class, etc.)
- Comments or identifiers

The Brute-Force String Matching Algorithm is the simplest method for searching a pattern. It compares the pattern with the text character by character without using any preprocessing or additional data structures.

Working Principle

Suppose,

- Text (T): `int total = amount + total;`
- Pattern (P): `total`

The algorithm starts from the first character of the text.

Step 1

Compare the pattern with the first position.

Text : `int total = amount + total;`

Pattern : `total`

First character doesn't match.

Move one position forward.

Step 2

Text : `int total = amount + total;`

^

Pattern : `total`

Again mismatch.

Move one character ahead.

Step 3

Continue comparing until the first character matches.

Text : int total = amount + total;

total

Pattern : total

Now compare each character.

t = t

o = o

t = t

a = a

l = l

All characters match.

Pattern found.

The algorithm continues searching the remaining text to find more occurrences.

Brute Force Algorithm

Input:

Text T

Pattern P

For i = 0 to n-m

j = 0

While j < m and T[i+j] == P[j]

j++

If j == m

Pattern Found

End

where

- n = length of text
- m = length of pattern

Flow of the Algorithm

Start



Read Text and Pattern



Compare Pattern with Current Position



Characters Match?



Yes → Compare Next Character



No → Shift Pattern by One Position



Entire Pattern Matched?

↓

Yes → Print Position

↓

Continue Until End of Text

↓

Stop

Example

Text

HELLO WORLD

Pattern

WORLD

Comparisons

HELLO WORLD

WORLD

Mismatch

HELLO WORLD

WORLD

Mismatch

HELLO WORLD

WORLD

Mismatch

HELLO WORLD

WORLD

Match Found

Pattern starts at position 6.

Applications in Code Editors

Brute-force searching is used for:

- Finding variable names
- Searching programming keywords
- Finding comments
- Locating class names
- Searching function names
- Replace operations in small files

b) Analyze its Time Complexity.

Let

- n = Length of text
- m = Length of pattern

Best Case

The first character mismatches every time.

Example

Text : AAAAAAAAAA

Pattern : BCD

Only one comparison is performed at each position.

Time Complexity

$$O(n)$$

Average Case

Some characters match while others mismatch.

Average number of comparisons lies between best and worst cases.

Time Complexity

$$O(nm)$$

Worst Case

Every comparison matches until the last character.

Example

Text : AAAAAAAAAAAAAAAAAA

Pattern : AAAAB

The algorithm compares almost every character repeatedly.

Number of comparisons

$$(n-m+1) \times m$$

Time Complexity

$$O(nm)$$

Space Complexity

The algorithm requires only a few variables.

No additional arrays are used.

Space Complexity

$$O(1)$$

Complexity Table

Case	Time Complexity
Best Case	$O(n)$
Average Case	$O(nm)$

Case Time Complexity

Worst Case $O(nm)$

Space Complexity $O(1)$

Why Worst Case Occurs

Consider

Text

AAAAAAAAAAAAAAAA

Pattern

AAAAB

The first four characters match every time.

The fifth character mismatches.

The algorithm shifts only one position and repeats all comparisons again.

This leads to a large number of unnecessary comparisons.

c) Justify the Need for Optimized String Searching Methods.

Although brute-force searching is simple, it becomes inefficient for large software projects.

Modern applications contain:

- Millions of lines of source code
- Thousands of variables
- Numerous libraries
- Large databases

Searching every character repeatedly wastes CPU time.

Therefore, optimized algorithms are preferred.

Limitations of Brute Force

1. Slow for Large Files

Large source files require many comparisons.

Performance decreases as file size increases.

2. Repeated Comparisons

The same characters are compared again and again.

This increases execution time.

3. Poor Scalability

As project size grows, search time increases significantly.

4. High CPU Usage

Continuous character comparisons consume processor resources.

5. Delayed Search Results

Editors become slower when searching huge projects.

Optimized String Searching Algorithms

1. Knuth–Morris–Pratt (KMP)

- Uses preprocessing of the pattern.
- Avoids unnecessary comparisons.
- Time Complexity:

$$O(n + m)$$

2. Boyer–Moore Algorithm

- Starts matching from the end of the pattern.
- Skips many characters.
- Very fast in practical applications.

3. Rabin–Karp Algorithm

- Uses hash values.

- Efficient for searching multiple patterns.
- Average Time Complexity:

$$O(n + m)$$

4. Aho–Corasick Algorithm

- Searches multiple keywords simultaneously.
- Used in compilers and antivirus software.

Comparison Table

Algorithm	Best Use	Time Complexity
Brute Force	Small text files	$O(nm)$
KMP	Single pattern search	$O(n+m)$
Boyer–Moore	Large documents	Very efficient in practice
Rabin–Karp	Multiple pattern search	$O(n+m)$ average
Aho–Corasick	Searching many keywords	$O(n + \text{total pattern length} + \text{matches})$

Real-World Applications

Optimized string searching is used in:

- Visual Studio Code
- IntelliJ IDEA
- Eclipse
- Compiler lexical analyzers
- Search engines
- Antivirus software
- DNA sequence analysis
- Text editors and IDEs
- Database query systems

Advantages of Optimized Algorithms

- Faster execution
- Fewer character comparisons
- Better performance on large files
- Lower CPU utilization
- Improved scalability
- Faster "Find" and "Replace" operations
- Better user experience in modern code editors

Conclusion

The Brute-Force String Matching Algorithm is simple, easy to implement, and suitable for small text files or introductory learning. However, its worst-case time complexity of $O(nm)$ makes it inefficient for searching in large software projects because it performs many repeated comparisons. Modern code editors therefore use optimized algorithms such as KMP, Boyer–Moore, Rabin–Karp, and Aho–Corasick, which significantly reduce search time and improve overall performance. These algorithms enable fast and efficient searching, making them essential for today's large-scale software development environments.